ROS2

DAY 6 – Manual 2

Parameters – Implementation

Overview – What This Manual Covers

In Manual 2, you will learn:

- Writing a parameter server node (Python)
- Declaring parameters
- Getting values
- Updating values dynamically using parameter callback
- Loading parameters from a YAML file
- Using a launch file to pass parameters
- Testing everything using ROS2 CLI

This is everything required for real robotics projects.

Final Expected Folder Structure

```
ros2_ws/
└── src/
    └── my_param_pkg/
        ├── my_param_pkg
        │   ├── __init__.py
        │   ├── param_node.py      <-- main node
        ├── params.yaml            <-- YAML parameters
        ├── launch
        │   └── param_launch.py    <-- launch file
        ├── package.xml
        ├── setup.py
        └── setup.cfg
```

PARAMETER NODE CODE (PYTHON)

param_node.py (Full Code, Fully Explained)

```
import rclpy
from rclpy.node import Node
from rclpy.parameter import Parameter


class ParameterNode(Node):
    def __init__(self):
        super().__init__('parameter_node')

        # ------------------------------
        # 1) DECLARE PARAMETERS
        # ------------------------------
        self.declare_parameter("speed", 1.0)
        self.declare_parameter("robot_name", "MY_ROBOT")
        self.declare_parameter("use_safety", True)
        self.declare_parameter("threshold", 10)

        # Print initial values
        self.print_current_parameters()

        # ------------------------------
        # 2) PARAMETER CALLBACK
        # ------------------------------
        self.add_on_set_parameters_callback(self.parameter_callback)

        # Timer just prints the speed every 2 seconds
        self.timer = self.create_timer(2.0, self.timer_callback)

    # ---------------------------------------------------------------------
    # GETTING PARAMETERS (Called once at startup)
    # ---------------------------------------------------------------------
    def print_current_parameters(self):
        speed = self.get_parameter("speed").value
        robot_name = self.get_parameter("robot_name").value
        use_safety = self.get_parameter("use_safety").value
        threshold = self.get_parameter("threshold").value

        self.get_logger().info(f"[STARTUP PARAMETERS]")
        self.get_logger().info(f"speed: {speed}")
        self.get_logger().info(f"robot_name: {robot_name}")
        self.get_logger().info(f"use_safety: {use_safety}")
        self.get_logger().info(f"threshold: {threshold}")
```

```
    # ------------------------------------------------------------------
    # 3) PARAMETER CALLBACK – DYNAMIC UPDATES
    # ------------------------------------------------------------------
    def parameter_callback(self, params):
        for param in params:
            self.get_logger().info(
                f"Parameter '{param.name}' updated to: {param.value}"
            )
        return rclpy.parameter.SetParametersResult(successful=True)

    # ------------------------------------------------------------------
    # 4) TIMER – Uses parameter live
    # ------------------------------------------------------------------
    def timer_callback(self):
        speed = self.get_parameter("speed").value
        self.get_logger().info(f"[TIMER] Current speed = {speed}")


def main(args=None):
    rclpy.init(args=args)
    node = ParameterNode()
    rclpy.spin(node)
    rclpy.shutdown()


if __name__ == "__main__":
    main()
```

COMPLETE EXPLANATION (line by line)

---

declare_parameter() – What EXACTLY Happens?

Example:

```
self.declare_parameter("speed", 1.0)
```

Internally ROS2:

1. Creates a parameter object
2. Assigns:
    - name → "speed"
    - type → double
    - value → 1.0

3. Registers into node's internal parameter server
4. Makes it available for:
    - launch params
    - YAML params
    - CLI modification

Parameter Callback – The Heart of Dynamic Parameters

```
self.add_on_set_parameters_callback(self.parameter_callback)
```

Meaning:

Whenever ANY parameter changes (by launch file or terminal),
this callback function automatically runs.

Callback receives a list of new values:

```
def parameter_callback(self, params):
    for param in params:
        print(param.name, param.value)
```

The function must return:

```
return SetParametersResult(successful=True)
```

Without this → parameter change is rejected.

YAML FILE

Create file:

params.yaml

```
parameter_node:
  ros__parameters:
    speed: 5.0
    robot_name: "TERMINATOR"
    use_safety: false
    threshold: 20
```

✓ Node name must match Python node name:

super().__init__('parameter_node')

So YAML uses:

parameter_node:

✓ All parameters go inside:

ros__parameters:

Done.

LAUNCH FILE (Loads YAML Automatically)

Create:

launch/param_launch.py

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package="my_param_pkg",
            executable="param_node",
            name="parameter_node",
            parameters=["params.yaml"]
        )
    ])
```

BUILD AND RUN

✓ Build

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

RUN USING THE LAUNCH FILE

ros2 launch my_param_pkg param_launch.py

Node will print loaded YAML parameters.

Test Parameter Updates (Dynamic change)

✓ Change speed:

ros2 param set /parameter_node speed 99.0

Terminal output:

Parameter 'speed' updated to: 99.0

✓ Check all parameters:

ros2 param list

✓ Get one value:

ros2 param get /parameter_node speed

Manual 2 Complete!